Tristan Jones
11/25/25
CSCI 8110

Project 4 GAN

1) Overview

This program is designed to implement a basic Generative Adversarial Network (GAN) to generate MNIST numbers. This is composed of a generator and a discriminator, where it follows the architecture specified in the assignment. This program essentially is guided by the discriminator learning to distinguish real MNIST images from generated ones and the generator learning to fool the discriminator over time.

2) Code Structure

Part 1 - Creating the Discriminator

```python
def build_discriminator():
    model = Sequential()

    model.add(Conv2D(filters=32, kernel_size=5, strides=(2, 2), padding="same",
                     input_shape=img_shape))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Dropout(0.4))

    model.add(Conv2D(64, kernel_size=5, strides=(2, 2), padding="same"))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Dropout(0.4))

    model.add(Conv2D(128, kernel_size=5, strides=(2, 2), padding="same"))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Dropout(0.4))

    model.add(Conv2D(256, kernel_size=5, strides=(2, 2), padding="same"))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Dropout(0.4))

    model.add(Flatten())
    model.add(Dense(1, activation="sigmoid"))  # scalar quality score in (0,1)

    img = Input(shape=img_shape)
    validity = model(img)

    return Model(img, validity)
```

Discriminator as described in the project requirements, essentially we are running through the stack of convolutions -> leakyReLU-> with a 40% dropout then flattened and passed into the dense sigmoid activation function.

```
def build_generator():
    model = Sequential()

    # Project and reshape
    model.add(Dense(7 * 7 * 192, input_dim=latent_dim))
    model.add(BatchNormalization(momentum=0.8))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Reshape((7, 7, 192)))

    # Upsample to 14x14
    model.add(Conv2DTranspose(96, kernel_size=5, strides=2, padding="same"))
    model.add(BatchNormalization(momentum=0.8))
    model.add(LeakyReLU(alpha=0.2))

    # Upsample to 28x28
    model.add(Conv2DTranspose(48, kernel_size=5, strides=2, padding="same"))
    model.add(BatchNormalization(momentum=0.8))
    model.add(LeakyReLU(alpha=0.2))

    # Additional layer
    model.add(Conv2DTranspose(24, kernel_size=5, strides=1, padding="same"))
    model.add(BatchNormalization(momentum=0.8))
    model.add(LeakyReLU(alpha=0.2))

    # Final conv to get 1 channel
    model.add(Conv2D(1, kernel_size=5, padding="same", activation="sigmoid"))

    noise = Input(shape=(latent_dim,))
    img = model(noise)

    return Model(noise, img)
```

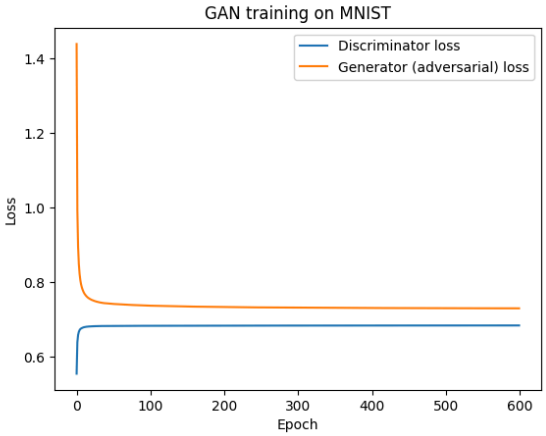
Generator as described in the project requirements we projecta 100-dimensional noise vector into a 7x7x192 tensor and then we run through the stack as outlined in the project specifications. In the end we get a 28x28x1 grayscale output from [0,1] which is out MNIST images.

```python
d_losses = []
g_losses = []

# Calculate batches per epoch
batches_per_epoch = num_samples // batch_size

# Pre-allocate arrays for speed
valid_label = np.ones((batch_size, 1), dtype=np.float32)  # No label smoothing
fake_label = np.zeros((batch_size, 1), dtype=np.float32)
valid_for_g = np.ones((batch_size, 1), dtype=np.float32)

print(f"\nStarting training: {epochs} epochs, {batches_per_epoch} batches per epoch")
print("-" * 60)

for epoch in range(1, epochs + 1):

    epoch_d_losses = []
    epoch_g_losses = []

    for batch in range(batches_per_epoch):
        # ---------------------
        #  Train Discriminator
        # ---------------------
        # Make discriminator trainable
        discriminator.trainable = True

        # Sample a random batch of real images
        idx = np.random.randint(0, num_samples, batch_size)
        real_imgs = x_train[idx]

        # Sample noise and generate a batch of fake images
        noise = np.random.normal(0, 1, (batch_size, latent_dim)).astype(np.float32)
        fake_imgs = generator(noise, training=False)

        # Train D on real and fake
        d_loss_real = discriminator.train_on_batch(real_imgs, valid_label)
        d_loss_fake = discriminator.train_on_batch(fake_imgs, fake_label)

        # Average discriminator loss
        d_loss = 0.5 * (d_loss_real[0] + d_loss_fake[0])

        # ---------------------
        #  Train Generator
        # ---------------------
        # Freeze discriminator
        discriminator.trainable = False

        noise = np.random.normal(0, 1, (batch_size, latent_dim)).astype(np.float32)
        g_loss = combined.train_on_batch(noise, valid_for_g)

        epoch_d_losses.append(d_loss)
        epoch_g_losses.append(g_loss)

        # Show progress every 50 batches (less frequent printing = faster)
        if batch % 50 == 0:
            print(f"Epoch {epoch}/{epochs} | Batch {batch}/{batches_per_epoch} | D loss: {d_loss:.4f} | G loss: {g_loss:.4f}", end='\r')
        elif batch == batches_per_epoch - 1:
            print(f"Epoch {epoch}/{epochs} | Batch {batch}/{batches_per_epoch} | D loss: {d_loss:.4f} | G loss: {g_loss:.4f}")  # Final batch gets newline

    # Store average losses for the epoch
    d_losses.append(np.mean(epoch_d_losses))
    g_losses.append(np.mean(epoch_g_losses))

    # Print epoch summary
    print(f">>> Epoch {epoch}/{epochs} COMPLETE | Avg D loss: {d_losses[-1]:.4f} | Avg G loss: {g_losses[-1]:.4f}")
    print("-" * 80)

    # Save image samples at specified epochs
    if epoch in save_interval:
        save_generated_images(epoch, generator)
        print(f"*** Saved images for epoch {epoch} ***\n")
```

Alot to read here but this is the training loop as specified in the lecture 17 slides, essentially training the disciminator and then freezing it to train the Generator.

3) Results

Loss Curves



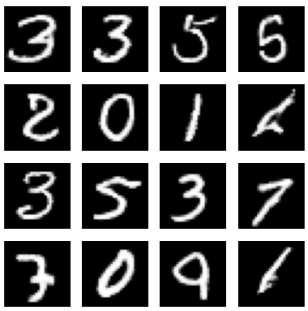
This showcases how the discriminator and generator losses evolve across 600 epochs it does get relatively close early on, but there is quite a lot of fine-tuning that happens between every singular epoch. Early on however, it begins very high for the generator ~1.45 which is expected since initially it really is just producing random noise until it can figure out what the discriminator knows. This graph showcases that they get to a relative equilibrium after about 50 epochs and indicates that neither model collapses or overwhelms the other. This suggests that this is a relatively balanced adversarial game, which is a sign of healthy GAN training.

Generated Images at Different Epochs:

Epoch 1:

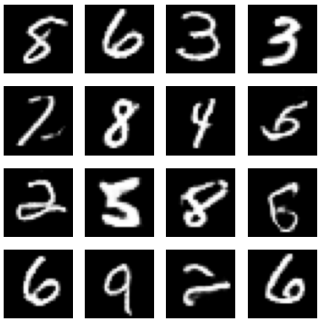
Very little coherence output really just looks like blobs and not really like numbers at all.

Epoch 200:



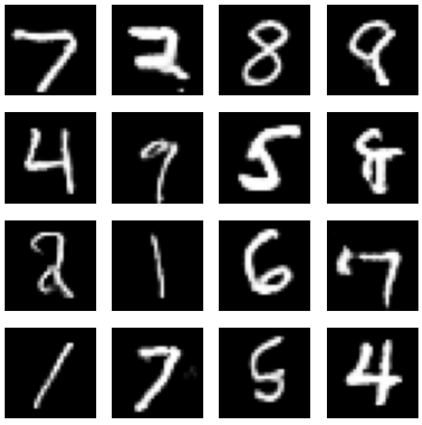
Here the numbers are signifigantrly more recognizable and clearly look like some kind of handwritten digits. However, it does also still look like there's some artifacts/overlapping strokes here and there.

Epoch 400:



Significantly sharper and much more well defined still some artifacts here and there but much sharper overall

Epoch 600:



I think 600 looks relatively similar to 400 but we can see that for a few of the numbers, specifically 1 and 9 its very sharp and well defined however the are still some problems it looks like with distinguishing 8 and 5.

4) Discussion

The goal of this project was to implement a basic GAN network using the provided architectures and to observe how adversarial training behaves on a standard dataset such as MNIST. I had a lot of difficulty at first in implementing this as I kept essentially just training and receiving mostly noise because I had somehow messed up the training states at first in the algorithm itself. But after a few days of tweaking the program and running it I did get it to operate as intended. The issue being that I needed to make sure to pause training on the discriminator when working with the generator. It does take about 12 hours to run, however, on my machine, as it goes through a lot of different cycles of training. I would love to mess around with this technology more in the future and see how it would work with training it on other types of pictures/objects, and not just handwritten numbers. I think that this would be a great means of future work for projects like this. Additionally, it would be interesting if training for longer would actually give better results, or what other methods I could use to implement to find better results for the digits themselves. As it stands between all of the different breaking points of 1, 200, 400, and 600 epochs, we did get varying degrees of increasing fidelity between all of these. Another interesting observation is that while the losses remained relatively stable for a long period of time the fidelity still did increase, meaning that the losses are really not entirely indicative of how well the model is learning. Overall, though the model eventually produced good-quality digits, this project highlighted how delicate adversarial training can be and how much careful tuning goes into getting a GAN to behave properly.

Resources Used:

[1] CSCI 8110 Lecture Notes, “Generative Adversarial Networks (Lecture 17),” University of Nebraska Omaha, 2025.

[2] “Assignment 4 – MNIST GAN Instructions,” CSCI 8110 Course Materials, University of Nebraska Omaha, 2025.

[3] TensorFlow Developers, “tf.keras.layers.Conv2D,” *TensorFlow API Documentation*, 2025. [Online]. Available: https://www.tensorflow.org/api_docs

[4] TensorFlow Developers, “tf.keras.Model,” *TensorFlow API Documentation*, 2025. [Online]. Available: https://www.tensorflow.org/api_docs

[5] TensorFlow Developers, “Keras Datasets – MNIST,” *TensorFlow API Documentation*, 2025. [Online]. Available: https://www.tensorflow.org/api_docs

[6] OpenAI, “ChatGPT 5.1,” OpenAI, 2025.

[7] GitHub, “GitHub Copilot,” GitHub, 2025.